

LeetCode 50: Pow(x, n)

Easy Python Recursive Fast Power Guide

Problem

Python

Recursion

$O(\log |n|)$

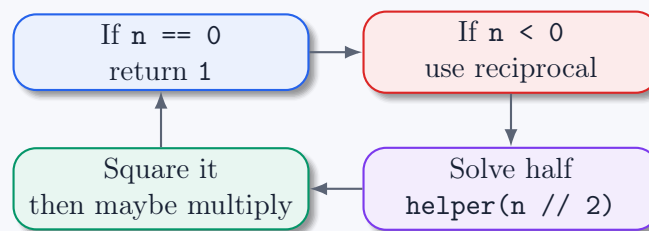
Goal. Write a Python function `myPow(x, n)` that returns x^n . Here, `x` is a float and `n` is an integer. The exponent can be positive, zero, or negative.

Main idea. Do not multiply by `x` one step at a time. That is too slow when `n` is large. Instead, cut the exponent in half again and again.

$$2^{10} = (2^5)^2$$

So if we can find the answer for half of the exponent, we can build the full answer with only one square.

1. The Simple Plan



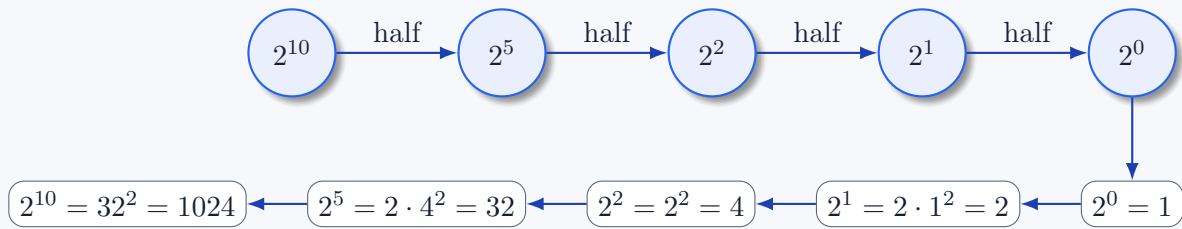
Ask this question: “If I already know $x^{n/2}$, how do I get x^n ?”

- If `n` is even: square the half answer. Example: $2^{10} = (2^5)^2$.
- If `n` is odd: square the half answer, then multiply by `x`. Example: $2^5 = 2 \cdot (2^2)^2$.
- If `n` is negative: solve the positive exponent, then flip it. Example: $2^{-3} = 1/2^3$.

2. Python Logic in Plain English

1. Make a helper function that only handles non-negative exponents.
2. Base case: if the exponent is 0, return 1.
3. Recursively compute `half = helper(exponent // 2)`.
4. Compute `squared = half * half`.
5. If the exponent is even, return `squared`.
6. If the exponent is odd, return `x * squared`.
7. In `myPow`, handle negative `n` by returning `1 / helper(-n)`.

3. Example: 2^{10}



4. Clean Recursive Python Solution

```
1 class Solution:
2     def myPow(self, x: float, n: int) -> float:
3         def helper(exponent: int) -> float:
4             if exponent == 0:
5                 return 1.0
6
7             half = helper(exponent // 2)
8             squared = half * half
9
10            if exponent % 2 == 0:
11                return squared
12            return x * squared
13
14        if n < 0:
15            return 1.0 / helper(-n)
16        return helper(n)
```

Why this is easy to implement: the helper only thinks about positive exponents. That keeps the recursion simple. The outer function handles the negative case once at the start.

5. Walk Through the Code

For `myPow(2, 10)`:

1. `helper(10)` asks for `helper(5)`.
2. `helper(5)` asks for `helper(2)`.
3. `helper(2)` asks for `helper(1)`.
4. `helper(1)` asks for `helper(0)`.
5. `helper(0)` returns 1.0.
6. Then each call returns upward: 2, 4, 32, 1024.

6. Edge Cases

Case	What happens
<code>n == 0</code>	Return 1.0. This is the base case.
<code>n < 0</code>	Return the reciprocal: <code>1.0 / helper(-n)</code> .
<code>x == 0</code> and <code>n > 0</code>	The recursion naturally returns 0.
<code>x == 1</code>	Always returns 1.
<code>x == -1</code>	Returns 1 for even <code>n</code> , and -1 for odd <code>n</code> .

7. Complexity

Time: $O(\log |n|)$
The exponent is halved each call.

Space: $O(\log |n|)$
Recursive call stack.

The algorithm is fast because each recursive call cuts the exponent in half:

$$10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

That is much shorter than multiplying ten times, and the difference becomes huge for large exponents.

8. Final Summary

To solve `Pow(x, n)` in Python, write a helper that computes powers for non-negative exponents. The helper returns 1.0 when the exponent is 0, recursively solves half of the exponent, squares that result, and multiplies by `x` only when the exponent is odd. For a negative exponent, compute the positive power and return its reciprocal.